PYTHON PROGRAMMING ASSIGNMENT REPORT

Course: CSA0823 – PYTHON PROGRAMMING

CO2 ASSIGNMENT 2

GITHUB LINK : https://github.com/Harish-S2026/CSA0823-Python-Programming

Submitted by

HARISH S
Register Number: 192512327

DEPT : ECE

QUESTION 1 – DECISIONS WITH MANY BOUNDARIES (6 MARKS)

1.1 Predictions before execution

The following eight households cover all required boundaries (100, 101, 200 and 201 units). The senior condition applies only above 200 units. A ration card is relevant only in the 101–200 slab.

Household	Units	Ration card	Member >80	Predicted decision
H1	100	No	No	Full subsidy
H2	101	Yes	No	Partial subsidy
H3	101	No	No	No subsidy
H4	150	Yes	No	Partial subsidy
H5	200	Yes	No	Partial subsidy
H6	200	No	Yes	No subsidy (senior exception is only above 200)
H7	201	No	Yes	Special senior subsidy
H8	201	Yes	No	No subsidy

1.2 Why separate if statements can give multiple results

Each if statement is tested independently. If two conditions overlap, both blocks can execute and append or print two decisions. In contrast, an if–elif–else chain tests alternatives in order and executes only the first matching branch. Therefore it gives one branch result. The separate-if implementation below uses mutually exclusive slab conditions, but a developer can accidentally write overlapping conditions such as units <= 200 and units >= 100.

1.3 Implementations and comparison

Implementation A uses separate if statements and collects results in a list. Implementation B uses if–elif–else and returns one decision. For the stated non-overlapping ranges, both should agree for all eight test cases. If the conditions are later broadened or written incorrectly, separate if statements can produce multiple outcomes.

See Task01/task01.py for executable versions, including a match–case alternative.

1.4 Design decision

Combined Boolean conditions are clearer for the stated rules because the important boundaries are visible in one expression: 101 <= units <= 200 and ration_card. A small nested if is still useful inside that slab to choose partial subsidy or no subsidy. Deep nesting would make the rule harder to scan. I would use an if–elif–else chain for the mutually exclusive consumption slabs and a short nested if for the additional eligibility condition.

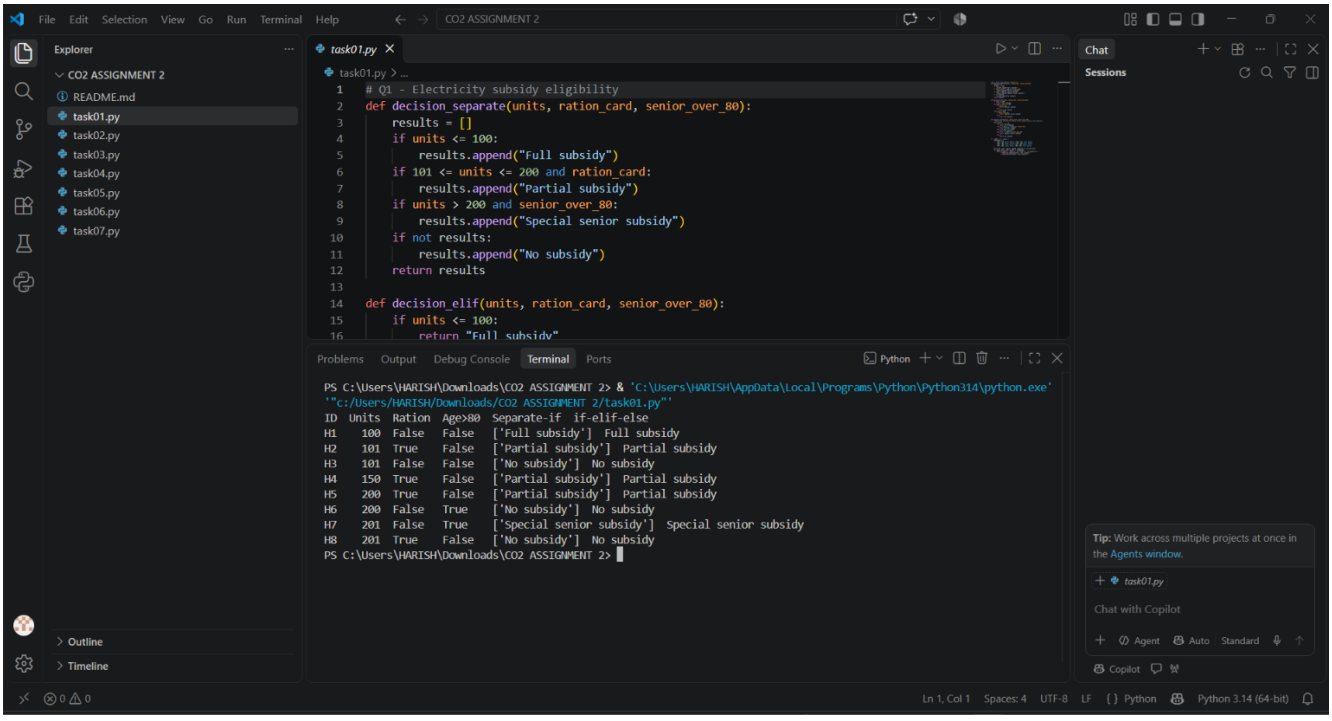
1.5 Match–case alternative and complexity

A match–case statement can use guarded cases: case x if x <= 100; case x if 101 <= x <= 200 and ration_card; case x if 101 <= x <= 200; case x if x > 200 and senior_over_80; and a final default case. The executable version is in Task01/task01.py. A lookup alternative stores ordered upper limits and decisions, then scans until a matching limit is found; conditions depending on ration-card or age still need a guard or a decision function.

For k slabs, an if–elif chain, guarded match–case, or linear lookup takes O(k) worst-case time and O(1) auxiliary space. If k is a fixed small number, this is effectively constant time. A dictionary lookup is average O(1) only when the key directly represents the complete decision state; it is less natural for numeric ranges.

1.6 Observed boundary conclusion

The four boundary outcomes are: 100 → full subsidy; 101 → partial only with a ration card; 200 → partial only with a ration card; 201 → special senior subsidy only when a member is above 80. The boundary checks prevent gaps and accidental overlap.



QUESTION 2 – LOOPS THAT COUNT ATTEMPTS (4 MARKS)

2.1 while-loop design

A while loop is natural when the number of attempts is controlled by a changing condition, such as attempts < limit and the OTP has not yet been accepted. Increment the counter only after a nonblank entry. A blank entry uses continue, so it does not consume an attempt. A correct OTP uses break.

2.2 for-loop design and loop–else

A for loop with range(3) is suitable when exactly three counted attempts are permitted. The loop's else clause runs only when the loop finishes without break, so it can display the lock message after three unsuccessful attempts. In production code, blank entries must not consume a range slot; Task02 demonstrates skipping blanks while retrieving the next nonblank entry.

2.3 break, continue, pass and else

- break: immediately exits the attempt loop after a correct OTP or after the lock condition is reached.
- continue: skips the rest of the current iteration for a blank entry; the attempt counter is unchanged.
- pass: performs no operation. It is useful temporarily when a block is syntactically required, but it is not needed for OTP control itself.
- for–else: the else block executes only if the loop was not terminated by break. It is suitable for the lock message after all allowed attempts fail.

2.4 Counter bug

If the counter is incremented before checking for a blank entry, blank submissions incorrectly consume attempts. If it is incremented after a possible break or not incremented at all, the loop may allow too many attempts or never reach the lock condition. Increment exactly once for each nonblank, incorrect-or-correct submitted OTP.

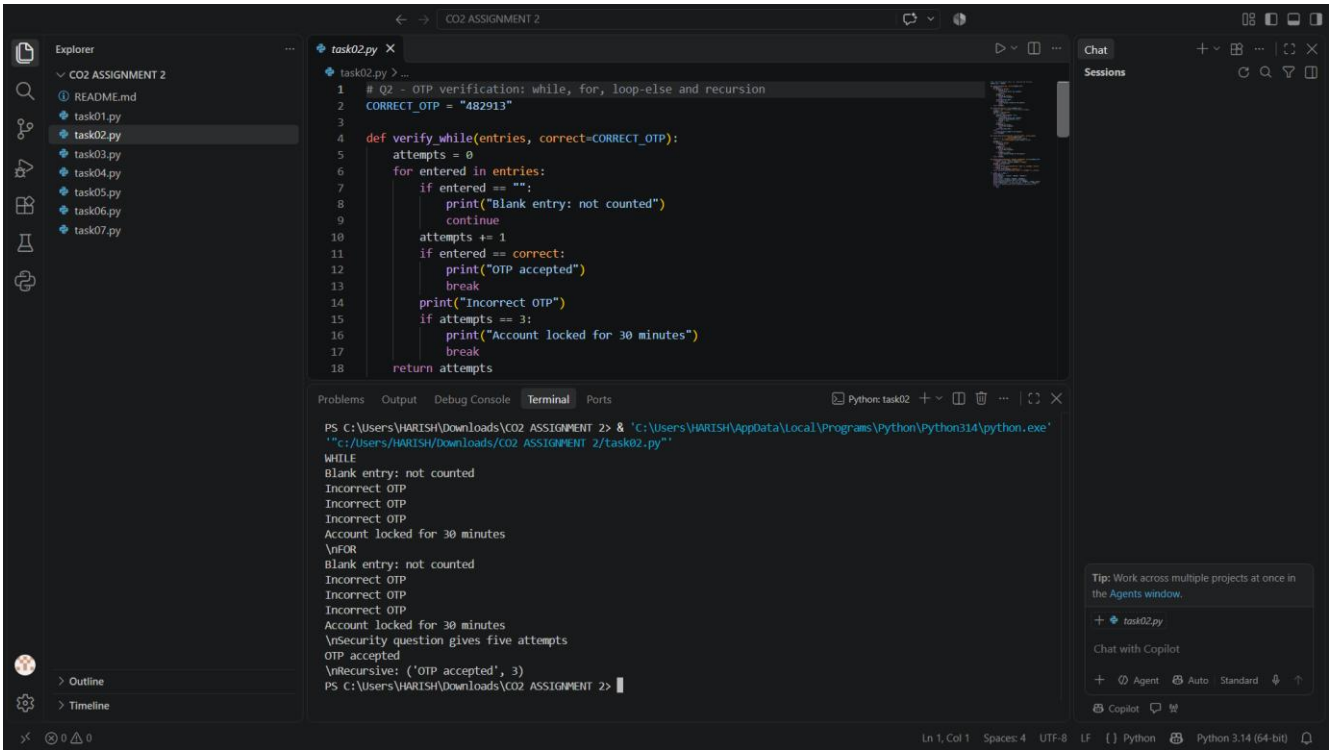
2.5 Changed condition: two extra attempts

Set the attempt limit to five only when the security answer is correct; otherwise retain three. The same loop structure can use limit = 3 + 2 when the security check succeeds. The loop should not duplicate the OTP logic. Task02 implements this condition.

2.6 Recursive alternative and complexity

A recursive function can process the next entry and call itself with the next index and current attempt count. It must stop when the OTP is correct, the limit is reached, or there are no entries. For a attempts, loop and recursive versions perform O(a) work. A loop uses O(1) auxiliary space; recursion uses O(a)

call-stack space and may hit Python's recursion limit. The loop is therefore preferable for a small, bounded verification workflow.



QUESTION 3 – VEHICLE REGISTRATION NUMBER VALIDATOR (6 MARKS)

3.1 Normalisation, validation and receipt formatting

Normalisation strips leading/trailing whitespace, removes spaces and hyphens, and converts letters to uppercase. The canonical structure is SS DD S[S] DDDD: two state letters, two district digits, one or two series letters, and four final digits. Validation checks the total length (9 or 10) and then checks each slice with isalpha() or isdigit(). No regular expression is used in the primary solution.

Input prediction	Normalised value	Prediction
tn 09 ab 1234	TN09AB1234	Valid
KA-01-M-0007	KA01M0007	Valid
Tn09Ab1234	TN09AB1234	Valid
TN9AB1234	TN9AB1234	Invalid: district has one digit
TN 22 1234 AB	TN221234AB	Invalid: series/final digits in wrong positions
MH 12 XY 9876	MH12XY9876	Valid

3.2 Complete-the-code solution

TODO 1: return text.strip().replace(' ', '').replace('-', '').upper(). TODO 2: reject lengths other than 9 or 10; check reg[0:2].isalpha(), reg[2:4].isdigit(), reg[4:-4].isalpha(), and reg[-4:].isdigit(); additionally ensure the series slice has length one or two. Task03 contains the complete executable program.

3.3 String immutability and upper()

Python strings are immutable: once created, their character sequence cannot be changed in place. This allows strings to be safely shared, hashed, and used as dictionary keys. text.upper() returns a new uppercase string; it does not modify text. To keep the result, assign it: text = text.upper().

3.4 Slicing explanation

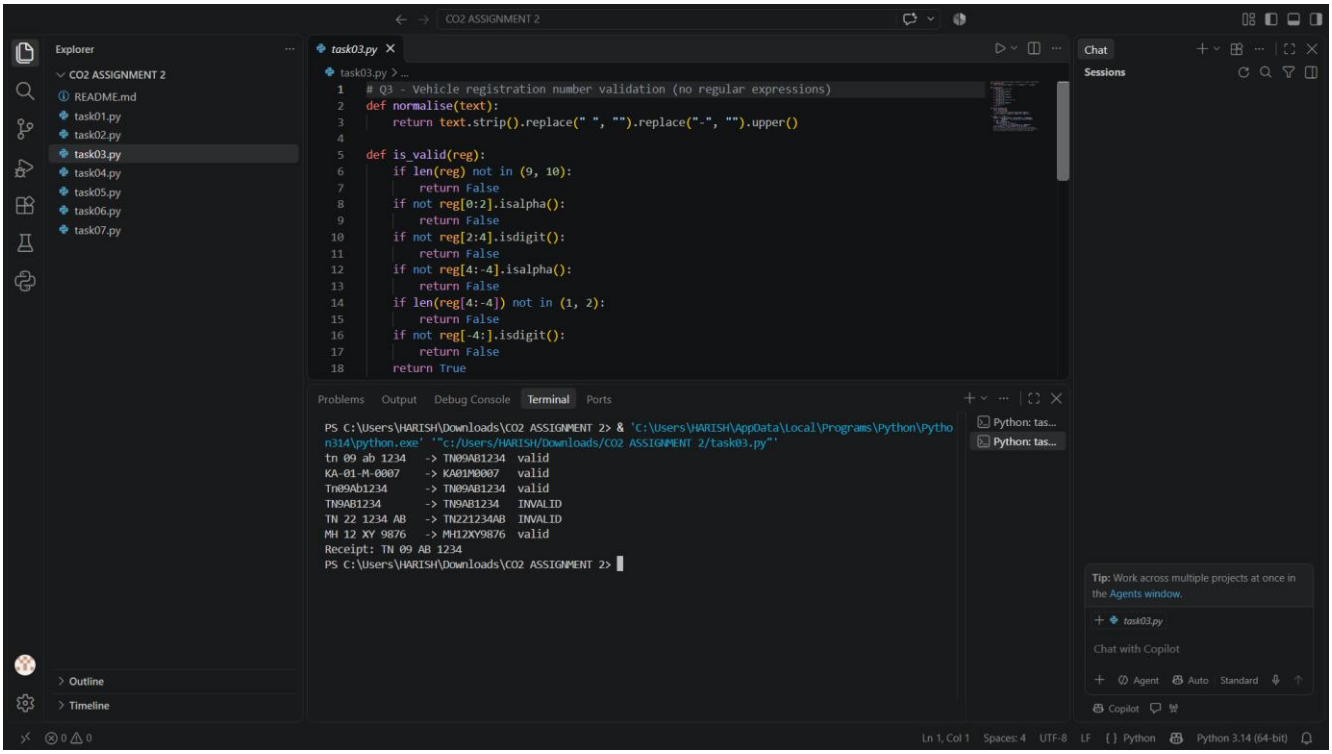
For a 9-character registration string, s[-4:] returns characters at positions 5–8 (the final four digits), and s[4:-4] returns the one-character series. For a 10-character string, s[-4:] still returns the final four characters, while s[4:-4] returns the two-character series. Slicing uses an exclusive stop index.

3.5 Regular-expression alternative and complexity

The alternative is re.fullmatch(r'[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}', reg). fullmatch requires the entire string to follow the pattern. Both the slicing implementation and the regular-expression implementation take O(L) time for a string of length L and O(L) space if normalisation creates a new string. Slicing is explicit and uses only basic string operations; a regular expression is compact and easier to extend when the format becomes more complex, but it requires importing re and understanding the pattern syntax.

3.6 Expected execution

The supplied four examples produce TN09AB1234 valid; KA01M0007 valid; TN9AB1234 INVALID; and TN221234AB INVALID. A valid receipt output is TN 09 AB 1234.



QUESTION 4 – BUILDING LONG TEXT: + OR JOIN()? (6 MARKS)

4.1 Three approaches and prediction

Repeated += appends each part to an immutable string. Conceptually, a new string may be allocated and previous characters copied on each iteration. A list of pieces followed by one join() performs a single final concatenation pass. An f-string is useful for formatting each line, but it does not by itself solve repeated concatenation; collecting formatted lines and joining them is the efficient pattern. I predict join() will be fastest or among the fastest for large input.

4.2 Complete-the-code solution

TODO 1: return ';'.join(parts) + (';' if parts else ''). This produces exactly the same trailing-semicolon format as the += version. Task04 includes +=, join(), and f-string-per-line construction, and benchmarks 1,000, 10,000 and 200,000 lines three times each.

4.3 Measured benchmark results in this report environment

The following are three-run averages measured while generating this report. They are illustrative, not universal. Re-run Task04 on the computer used for the assignment and report those local measurements because CPU, Python build, memory and background load affect timings.

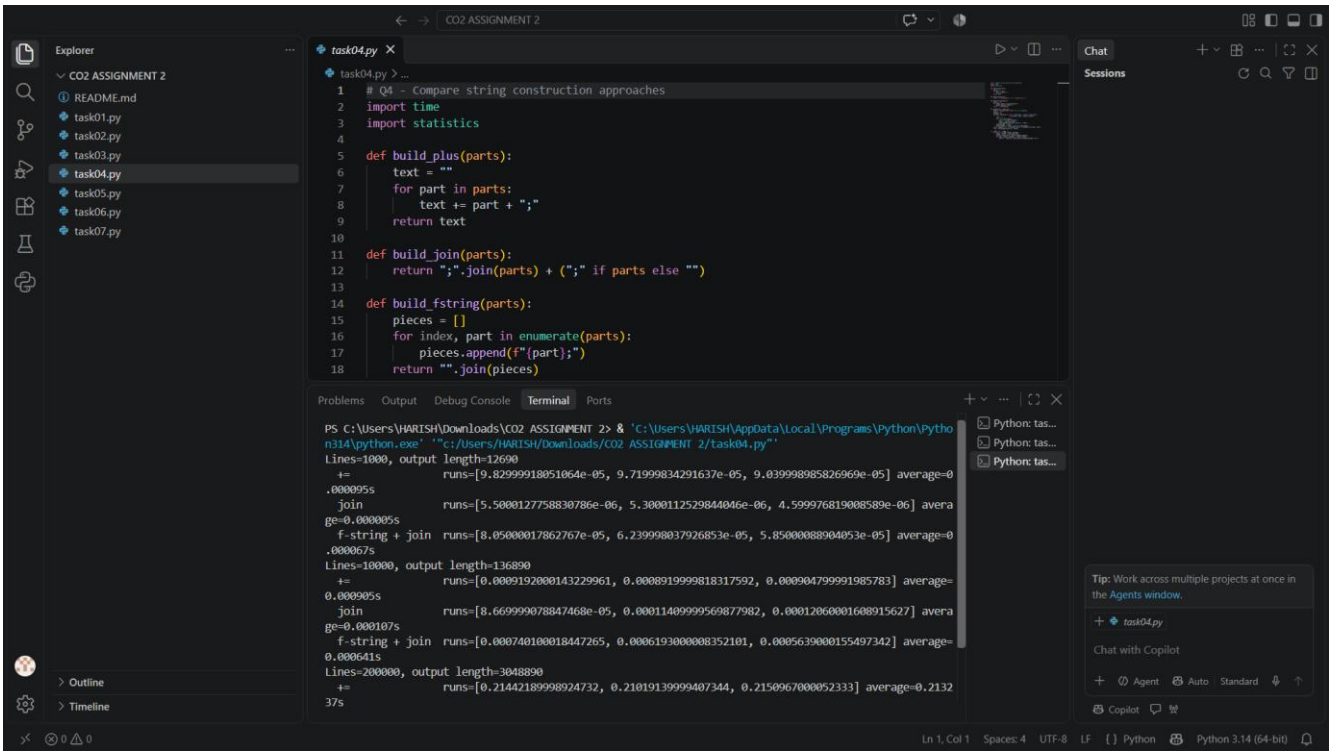
Lines	Output length	+= average (s)	join average (s)	f-string + join average (s)
1000	12690	0.000108	0.000023	0.000110
10000	136890	0.001092	0.000108	0.000814
200000	3048890	0.025451	0.005702	0.030505

4.4 Immutability and CPython optimisation

Because strings are immutable, concatenation conceptually creates a new string rather than modifying the existing one. Repeated concatenation can therefore copy an increasing amount of text and may approach $O(N^2)$ character-copying work in the general case. CPython sometimes optimises local += operations when it can safely resize the string, so measured behaviour can be better than the simple worst-case model. This optimisation is implementation-dependent and should not be relied upon. join() is recommended because it expresses the intended operation directly and has O(N) total character-processing time for N output characters.

4.5 Alternative: file streaming or StringIO

For very large statements that do not need to remain entirely in memory, open a text file and write each formatted line as it is generated. This uses O(1) additional application memory (apart from buffering) and O(N) total output work. io.StringIO provides a file-like in-memory buffer; write each line and call getvalue() at the end. It uses O(N) memory for the final content but avoids repeatedly rebuilding the whole string. A list plus join also uses O(N) memory for the parts and final result. Streaming to a file is preferable when the output is large and can be consumed as a file.



QUESTION 5 – FUNCTIONS, PARAMETERS AND SCOPE (7 MARKS)

5.1 Mutable default argument bug

In add_item_buggy(item, order=[]), the list is created once when Python defines the function. Every call that omits order uses that same list object. Appending Tea and then Vada therefore makes both returned references show ['Tea', 'Vada']. The corrected pattern uses order=None, creates a new list inside the function when order is None, appends the item and returns the list.

5.2 Complete-the-code and expected output

TODO 1: if order is None: order = []; then order.append(item); return order. The expected output is ['Tea', 'Vada'] ['Tea', 'Vada'] for the buggy function, followed by ['Tea'] ['Vada'] for the corrected function. add_item_buggy.__defaults__ displays a tuple containing the single shared list object; after the calls its contents are ['Tea', 'Vada'].

5.3 Parameter types

- Positional parameters: use when argument order is obvious, such as add_item('Tea', order).
- Keyword parameters: use when clarity matters, such as calculate_bill(amount=200, gst_rate=0.05).
- Default parameters: use for optional values with safe immutable defaults (None, numbers, strings, tuples). Avoid mutable list/dict defaults unless sharing is intentional.

5.4 Returning multiple values

A function can return a tuple: return total, gst. The caller can unpack it as total, gst = calculate_bill(200). Python functions can return any object, so a dictionary or a small data class is also possible when named fields are useful.

5.5 Local, global and nonlocal

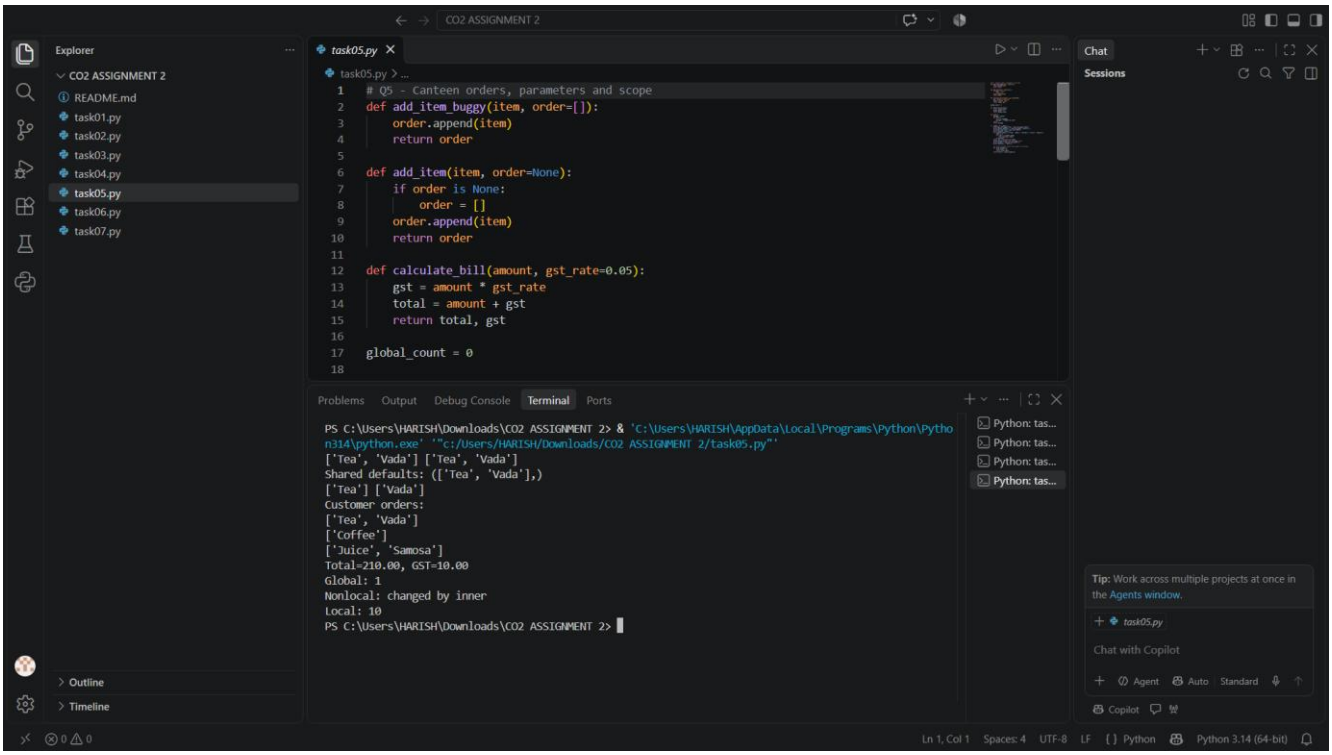
A local name is created inside a function and is available in that function's scope. A global name is defined at module level; assigning to it inside a function requires the global keyword. A nonlocal name refers to a variable in the nearest enclosing function scope; nonlocal allows an inner function to rebind that enclosing variable. Task05 demonstrates all three.

Changing globals is usually avoided because it creates hidden dependencies: any function can alter shared state, making testing and debugging harder. Passing values as parameters and returning results makes dependencies explicit.

5.6 Three-customer test and alternative designs

Customer	Items	Independent result
Customer 1	Tea, Vada	['Tea', 'Vada']
Customer 2	Coffee	['Coffee']
Customer 3	Juice, Samosa	['Juice', 'Samosa']

If the caller always passes an explicit list, add_item(item, order) simply appends to that caller-owned list; this is clear and uses O(1) amortised time per append. A closure can create and retain a private order list for one customer, returning an add function that appends to it. The closure uses O(m) space for m items and keeps customer state private. The None-default design creates a fresh list per independent call when no list is supplied; the list storage is O(m) for m items. All three designs take O(1) amortised time per append. The correct choice depends on who should own and manage the order list.



QUESTION 6 – RECURSION AND ITS LIMITS (8 MARKS)

6.1 Recurrence and base cases

Let W(n) be the number of ways to climb n steps using jumps of one or two. For n <= 1, W(n) = 1 (one way to stand at the start or take one step). For n >= 2, W(n) = W(n-1) + W(n-2), because the final move is either one step from n-1 or two steps from n-2.

6.2 Hand calculations

n	Ways W(n)
1	1
2	2
3	3
4	5
5	8
6	13

6.3 Why plain recursion repeats work

For n=5, the call tree begins W(5) → W(4), W(3); W(4) → W(3), W(2); W(3) → W(2), W(1). W(3) and W(2) are evaluated repeatedly in different branches. This duplicated work causes exponential growth. The number of calls for the naive recurrence follows approximately $O(\varphi^n)$, where φ is the golden ratio.

A compact view of the call tree is: W(5) [W(4) [W(3) [W(2), W(1)], W(2)], W(3) [W(2), W(1)]]. Repeated labels represent separate recalculations.

6.4 Memoisation and call counts

Memoisation stores each computed W(n) and reuses it. With lru_cache, each distinct n is calculated once. The first call to W(25) therefore has 26 cache-miss function calls for n = 0 through 25, producing the same answer 121393 as naive recursion, which makes 242785 total calls.

n	Ways	Naive calls	Memoised calls (fresh cache)
10	89	177	11
20	10946	21891	21
25	121393	242785	26
30	1346269	2692537	31

6.5 n = 1,000 and recursion limit

The memoised recurrence still has recursion depth proportional to n. Python's default recursion limit is commonly around 1,000 (check sys.getrecursionlimit() on the actual machine), so ways_memo(1000) may raise RecursionError before reaching the base case. Memoisation removes repeated computation but does not remove recursive call-stack depth.

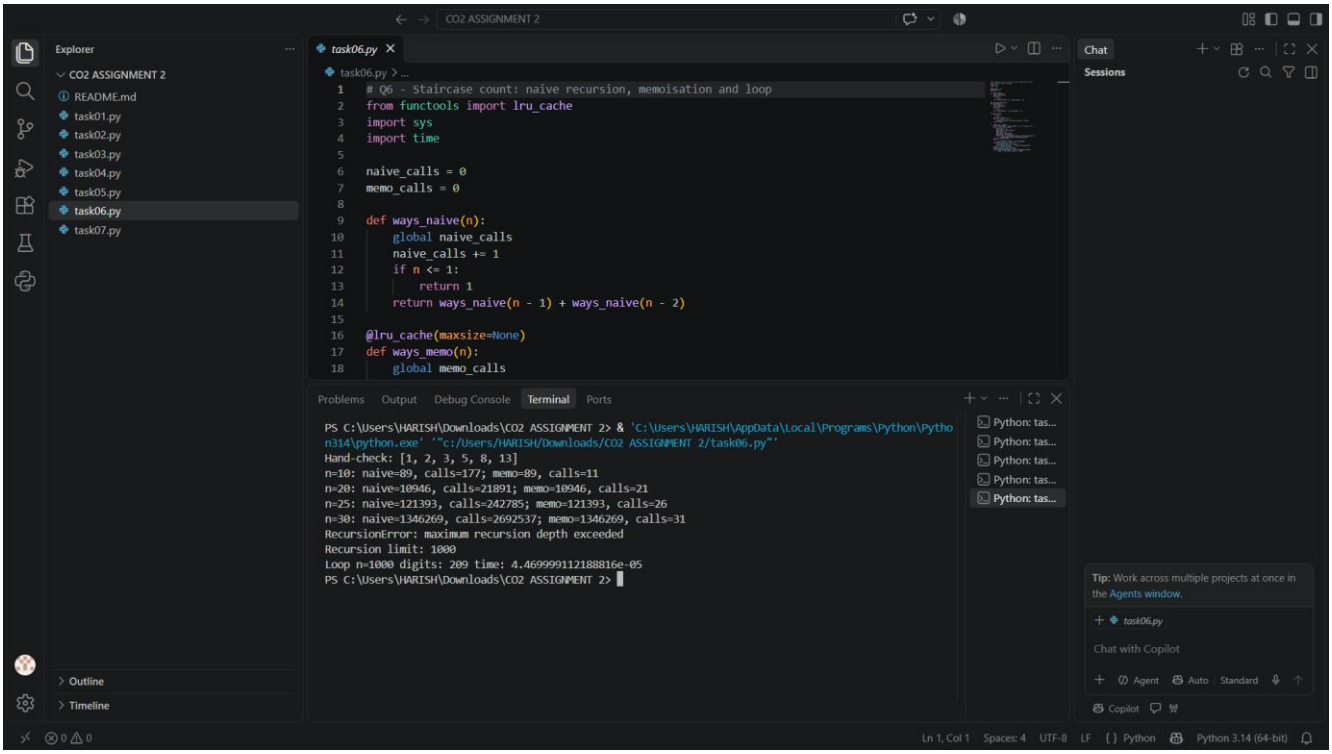
6.6 Deployment decision and iterative solution

For n up to 1,000, I would deploy the iterative two-variable solution. It computes the same recurrence in O(n) time and O(1) auxiliary space, and it avoids recursion-depth risk. Raising the recursion limit is not my primary solution because an excessive limit can risk stack or process failure. Memoisation remains useful when recursive structure is important or many overlapping subproblems are queried, but the loop is simpler and safer for this one-dimensional recurrence.

6.7 Complexity comparison

Method	Time	Auxiliary space	Main limitation
Plain recursion	$O(\varphi^n)$	O(n) stack	Repeated work; slow
Memoised recursion	O(n)	O(n) cache + O(n) stack	Recursion depth
Iterative two-value loop	O(n)	O(1)	No significant drawback for this recurrence

All three methods can compute the mathematical answer for n=1,000 in principle, but plain recursion is computationally impractical. Memoised recursion may hit RecursionError. The iterative loop handles n=1,000 safely. Python integers have arbitrary precision, so the answer grows in digit count; arithmetic cost is not strictly constant for very large integers.



QUESTION 7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

7A. Diagnose the bottleneck

Selected program: Q6 naive recursive staircase counter. The dominant work is repeated evaluation of the same subproblems W(n-1) and W(n-2). The number of recursive function calls grows exponentially with n. Evidence: at n=25 the naive version makes 242,785 calls; at n=30 it makes 2,692,537 calls, although the answer is only one integer. The repeated-call count, not input storage, is the bottleneck.

7B. Measurable design objective

Objective: preserve exactly the same answer for every tested n while reducing the number of algorithmic steps to at most n-1 loop iterations and using O(1) auxiliary storage. The redesigned program must also compute n=1,000 without RecursionError.

7C. Revised design and property exploited

The recurrence depends only on the two previous values. Therefore the full call tree and cache are unnecessary: keep only W(n-2) and W(n-1), calculate their sum, and shift the two variables forward. This exploits the recurrence's limited dependency window. No extra list or dictionary is required; two integer references are maintained. The trade-off is that the implementation is iterative rather than directly mirroring the mathematical recursive definition.

Pseudocode:

```
IF n <= 1: RETURN 1
previous = 1; current = 1
FOR step from 2 through n:
    next_value = previous + current
    previous = current
    current = next_value
RETURN current
```

7D. Prediction before implementation

Prediction: the iterative version will return the same result, reduce work from exponential recursive calls to n-1 iterations, and complete n=1,000 without recursion-depth failure. Its auxiliary space will remain O(1), excluding the memory required to store the growing integer.

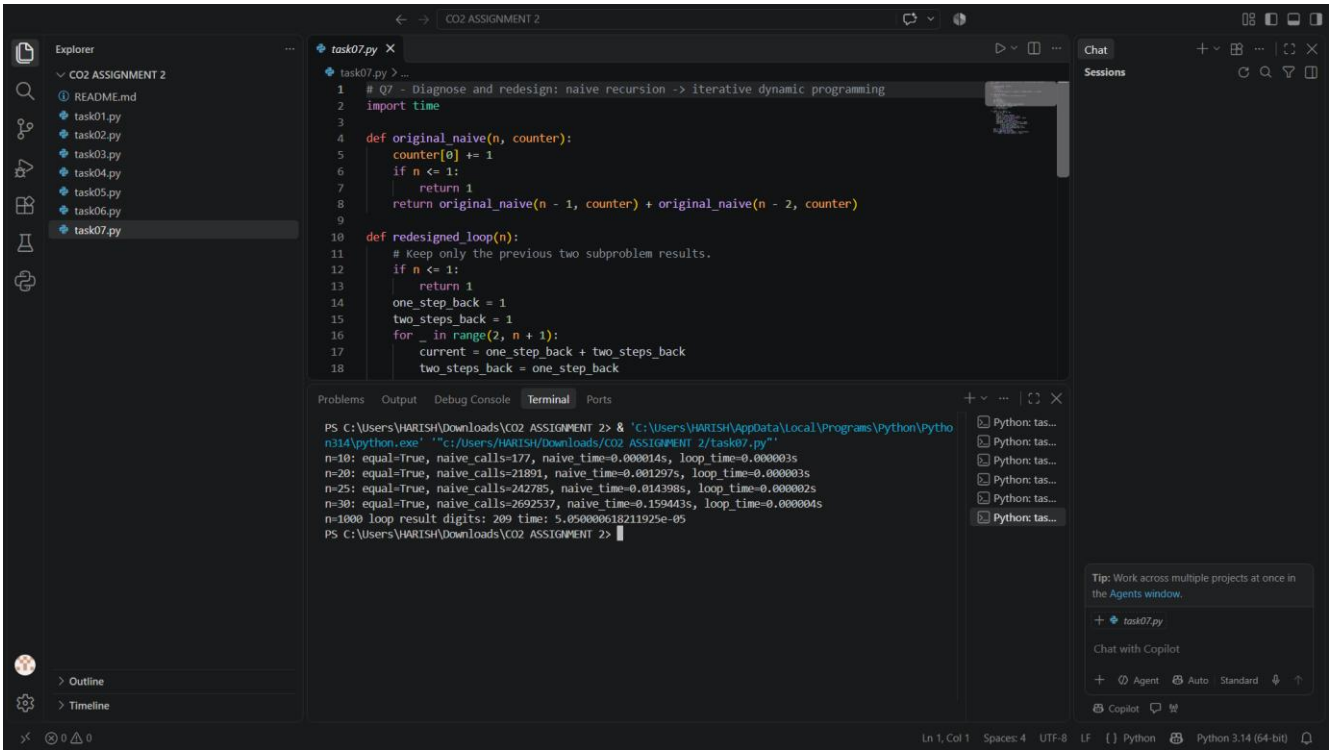
7E. Implementation and validation

Task07/task07.py contains the original naive implementation with a call counter and the redesigned loop. Both are run on identical n values. The following timing figures are measured in this report environment; timings vary, so rerun on the student's own computer. The call-count comparison is the stronger, hardware-independent evidence.

n	Result	Naive calls	Naive time (s)	Loop time (s)	Same result
10	89	177	0.000015	0.000003	True
20	10946	21891	0.001439	0.000002	True
25	121393	242785	0.017475	0.000008	True
30	1346269	2692537	0.181831	0.000010	True

7F. Final defence

The objective is achieved if the loop matches every result and uses a linear number of iterations. At n=25, 242,785 naive calls are replaced by 24 loop iterations; at n=30, 2,692,537 calls are replaced by 29 iterations. The redesigned solution also handles n=1,000 without recursion. The new cost is that the iterative code is less directly recursive in appearance, but it is short and easy to test. The original recursive version remains useful for teaching the recurrence and for very small n, but it is not suitable for larger staircases. This redesign is not merely a built-in replacement: it uses the recurrence's two-value dependency to remove repeated work.


CO2 ASSIGNMENT 2
Explorer
CO2 ASSIGNMENT 2
README.md
task01.py
task02.py
task03.py
task04.py
task05.py
task06.py
task07.py
Outline
Timeline
task07.py
task07.py > ...
Q7 - Diagnose and redesign: naive recursion -> iterative dynamic programming
import time
def original_naive(n, counter):
counter[0] += 1
if n <= 1:
return 1
return original_naive(n - 1, counter) + original_naive(n - 2, counter)
def redesigned_loop(n):
Keep only the previous two subproblem results.
if n <= 1:
return 1
one_step_back = 1
two_steps_back = 1
for _ in range(2, n + 1):
current = one_step_back + two_steps_back
two_steps_back = one_step_back
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\HARISH\Downloads\CO2 ASSIGNMENT 2> & 'C:\Users\HARISH\AppData\Local\Programs\Python\Python314\python.exe' '"c:/Users/HARISH/Downloads/CO2 ASSIGNMENT 2/task07.py"'
n=10: equal=True, naive_calls=177, naive_time=0.000014s, loop_time=0.000003s
n=20: equal=True, naive_calls=21891, naive_time=0.001297s, loop_time=0.000003s
n=25: equal=True, naive_calls=242785, naive_time=0.014398s, loop_time=0.000002s
n=30: equal=True, naive_calls=2692537, naive_time=0.159443s, loop_time=0.000004s
n=1000 loop result digits: 209 time: 5.050000618211925e-05
PS C:\Users\HARISH\Downloads\CO2 ASSIGNMENT 2>
Python: tas...
Python: tas...
Python: tas...
Python: tas...
Python: tas...
Python: tas...
Chat
Sessions
Tip: Work across multiple projects at once in the Agents window.
task07.py
Chat with Copilot
Agent
Auto
Standard
Copilot
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14 (64-bit)